

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
Факультет физико-математических и естественных наук

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 7

Студент: Николаева Ангелина

Борисовна

Студенческий билет:

1032253612

Группа: НКАбд-04-25

МОСКВА

2026

Содержание

1. Цель работы.....	4
2. Выполнение лабораторной работы.....	5
2.1. Реализация переходов в NASM	5
2.2. Изучение структуры файла листинга.....	10
2.3. Задания для самостоятельной работы.....	12
3. Выводы	17

Список иллюстраций

1.	Создание каталога и файла для программы	5
2.	Сохранение программы	6
3.	Запуск программы	6
4.	Изменение программы.....	7
5.	Запуск измененной программы.....	7
6.	Изменение программы.....	8
7.	Проверка изменений.....	8
8.	Сохранение новой программы.....	9
9.	Проверка программы из листинга.....	9
10.	Проверка файла листинга.....	10
11.	Удаление операнда из программы.....	11
12.	Просмотр ошибки в файле листинга.....	11
13.	Первая программа самостоятельной работы.....	12
14.	Проверка работы первой программы.....	14
15.	Вторая программа самостоятельной работы.....	14
16.	Проверка работы второй программы.....	16

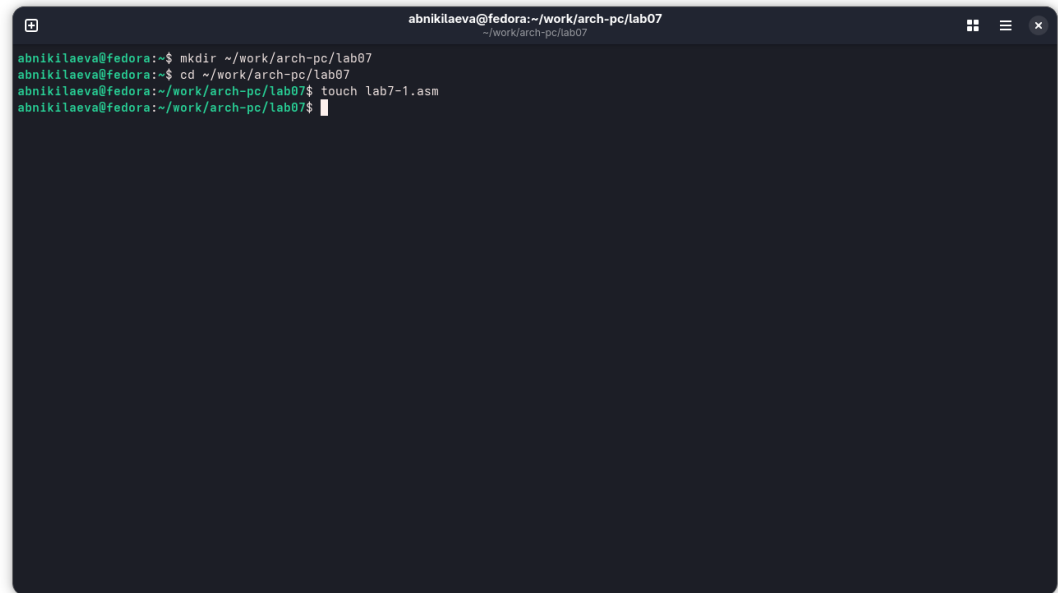
1.Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

2. Выполнение лабораторной работы

2.1. Реализация переходов в NASM

Создаю каталог для программ лабораторной работы №7 (рис. 2.1).

A screenshot of a terminal window with a dark background. The window title is 'abnikilaeva@fedora:~/work/arch-pc/lab07'. The terminal shows the following commands and their outputs: 'mkdir ~/work/arch-pc/lab07', 'cd ~/work/arch-pc/lab07', and 'touch lab7-1.asm'. The prompt is 'abnikilaeva@fedora:~/work/arch-pc/lab07\$' with a cursor at the end.

```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~$ mkdir ~/work/arch-pc/lab07
abnikilaeva@fedora:~$ cd ~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ touch lab7-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Рис. 2.1: Создание каталога и файла для программы

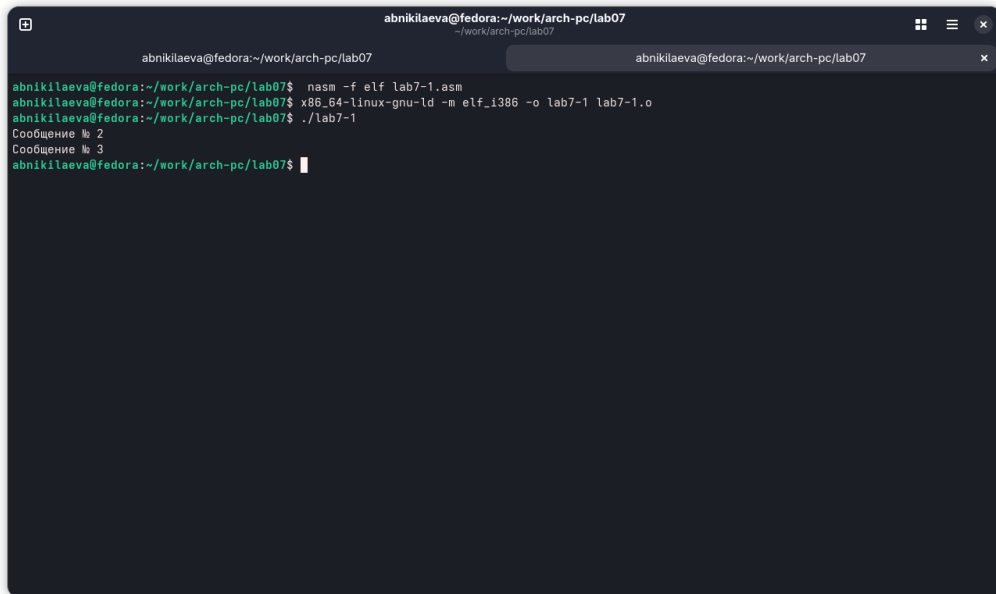
Копирую код из листинга в файл будущей программы. (рис. 2.2).



```
GNU nano 8.5 lab7-1.asm
#include 'in_out.asm'; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start
_start:
jmp _label2
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.2: Сохранение программы

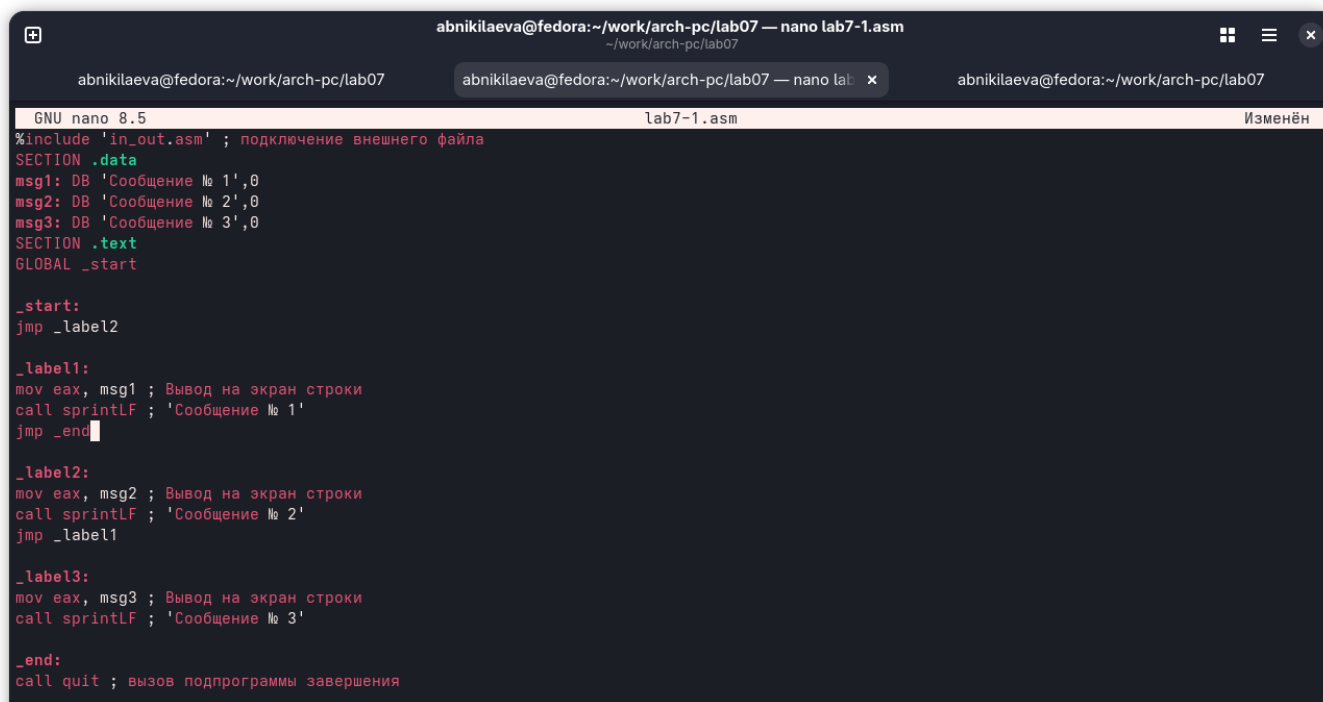
При запуске программы я убедился в том, что безусловный переход действительно изменяет порядок выполнения инструкций (рис. 2.3).



```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ x86_64-linux-gnu-ld -m elf_i386 -o lab7-1 lab7-1.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 3
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Рис. 2.3: Запуск программы

Изменяю программу таким образом, чтобы поменялся порядок выполнения функций (рис. 2.4).



```
GNU nano 8.5 lab7-1.asm Изменён
%include 'in_out.asm' ; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start

_start:
jmp _label2

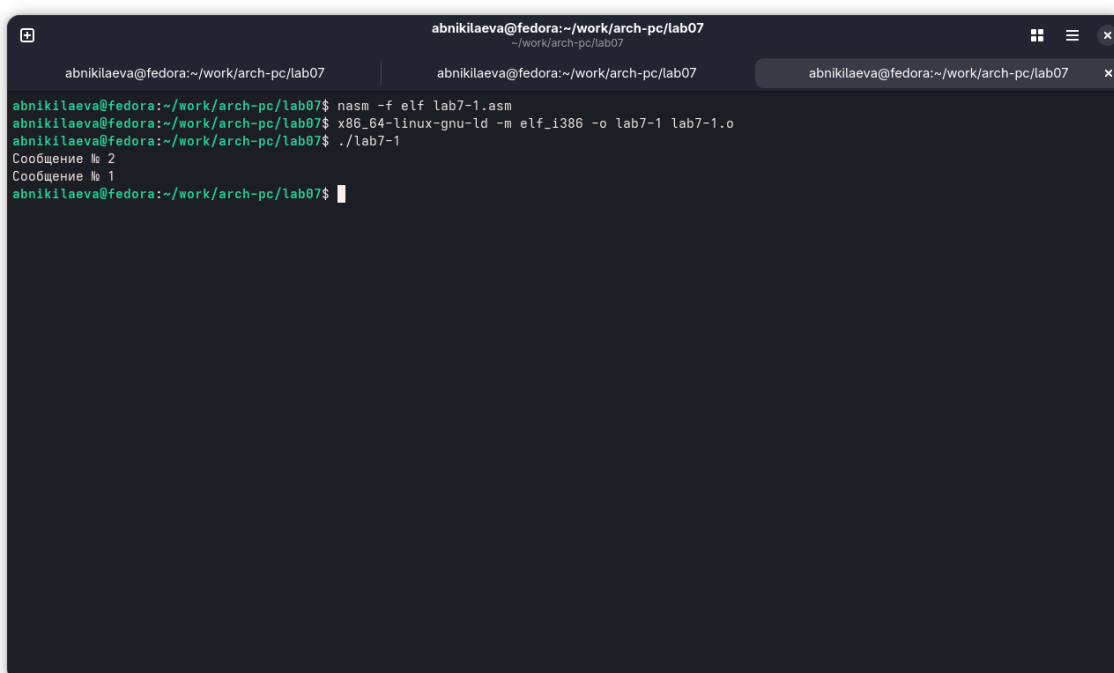
_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end

_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1

_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'

_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.4: Изменение программы



```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ x86_64-linux-gnu-ld -m elf_i386 -o lab7-1 lab7-1.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 2
Сообщение № 1
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Запускаю программу и проверяю, что примененные изменения верны (рис.2.5).

Рис. 2.5: Запуск измененной программы

Теперь изменяю текст программы так, чтобы все три сообщения вывелись в обратном порядке (рис. 2.6).



```
GNU nano 8.5 lab7-1.asm
#include 'in_out.asm'; подключение внешнего файла
SECTION .data
msg1: DB 'Сообщение № 1',0
msg2: DB 'Сообщение № 2',0
msg3: DB 'Сообщение № 3',0
SECTION .text
GLOBAL _start

_start:
jmp _label3

_label1:
mov eax, msg1 ; Вывод на экран строки
call sprintf ; 'Сообщение № 1'
jmp _end

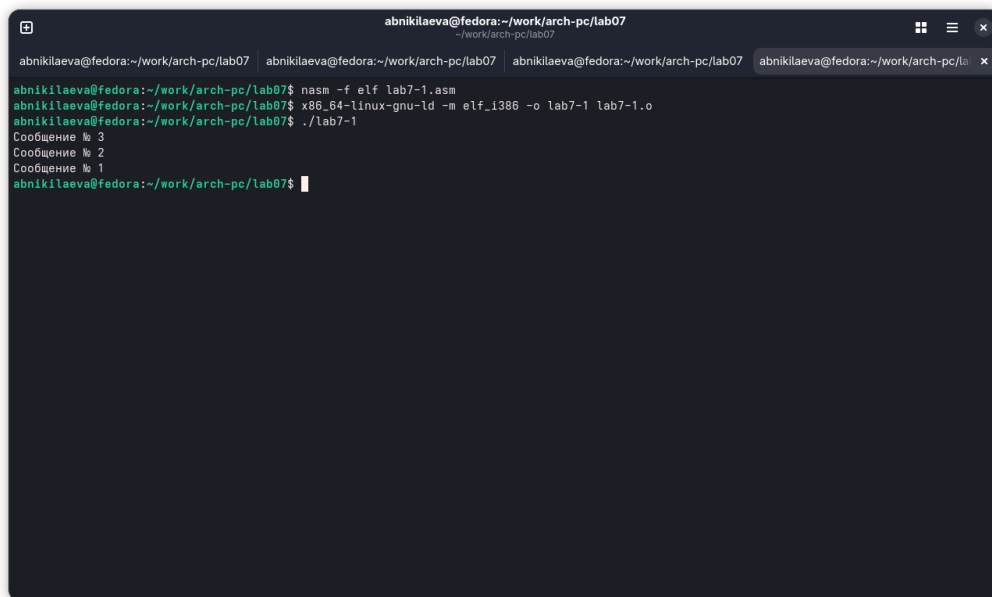
_label2:
mov eax, msg2 ; Вывод на экран строки
call sprintf ; 'Сообщение № 2'
jmp _label1

_label3:
mov eax, msg3 ; Вывод на экран строки
call sprintf ; 'Сообщение № 3'
jmp _label2

_end:
call quit ; вызов подпрограммы завершения
```

Рис. 2.6: Изменение программы

Работа выполнена корректно, программа в нужном мне порядке выводит сообщения

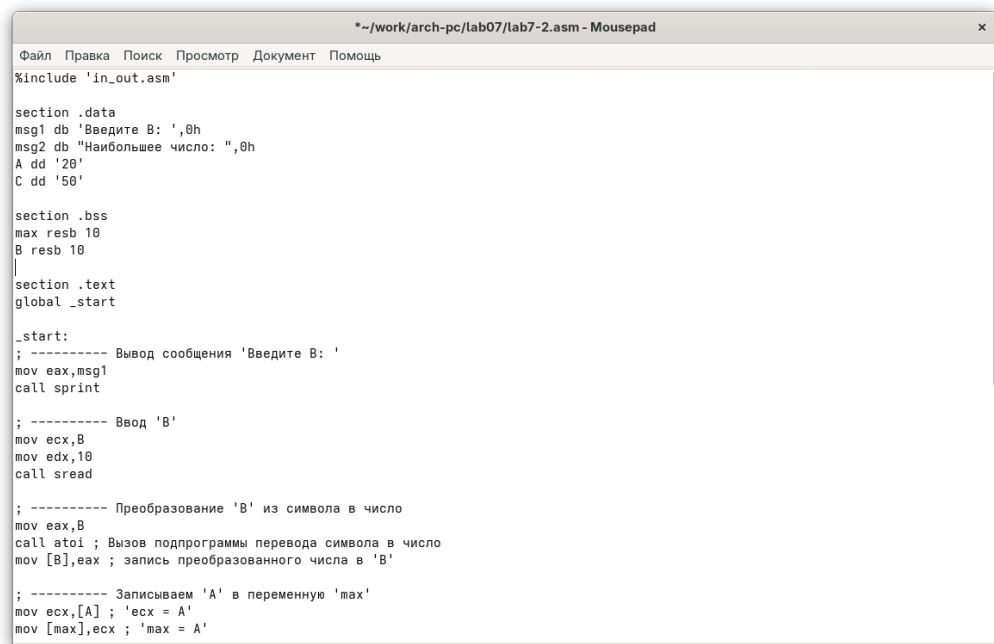


```
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-1.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ ld -m elf_i386 -o lab7-1 lab7-1.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-1
Сообщение № 3
Сообщение № 2
Сообщение № 1
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

(рис.2.7).

Рис. 2.7: Проверка изменений

Создаю новый рабочий файл и вставляю в него код из следующего листинга (рис. 2.8).



```
File Edit Search View Document Help
%include 'in_out.asm'

section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'

section .bss
max resb 10
B resb 10
|
section .text
global _start

_start:
; ----- Вывод сообщения 'Введите B: '
mov eax,msg1
call sprint

; ----- Ввод 'B'
mov ecx,B
mov edx,10
call sread

; ----- Преобразование 'B' из символа в число
mov eax,B
call atoi ; Вызов подпрограммы перевода символа в число
mov [B],eax ; запись преобразованного числа в 'B'

; ----- Записываем 'A' в переменную 'max'
mov ecx,[A] ; 'ecx = A'
mov [max],ecx ; 'max = A'
```

Рис. 2.8: Сохранение новой программы

Программа выводит значение переменной с максимальным значением, проверяю работу программы с разными входными данными (рис. 2.9).



```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ touch lab7-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ mousepad lab7-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ x86_64-linux-gnu-ld -m elf_1386 -o lab7-2 lab7-2.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 60
Наибольшее число: 60
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 30
Наибольшее число: 50
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 10
Наибольшее число: 50
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Рис. 2.9: Проверка программы из листинга

2.2. Изучение структуры файла листинга

Создаю файл листинга с помощью флага -l команды nasm и открываю его с помощью текстового редактора mousepad (рис. 2.10).

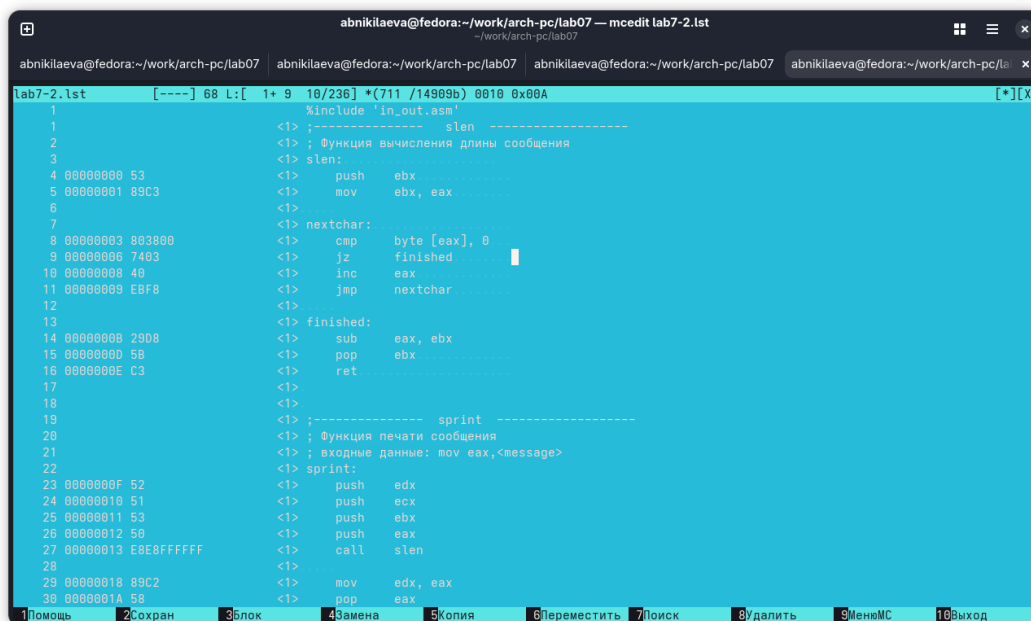


Рис. 2.10: Проверка файла листинга

Первое значение в файле листинга - номер строки, и он может вовсе не совпа-дять с номером строки изначального файла. Второе вхождение - адрес, смещение машинного кода относительно начала текущего сегмента, затем непосредственно идет сам машинный код, а заключает строку исходный текст прогарммы с комментариями.

Удаляю один операнд из случайной инструкции, чтобы проверить поведение файла листинга в дальнейшем (рис. 2.11).

```

abnikilaeva@fedora:~/work/arch-pc/lab07
~/work/arch-pc/lab07

abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07

14 global _start
15
16 _start:
17 ; ----- Вывод сообщения 'Введите B: '
18 mov eax,msg1
19 call sprint
20
21 ; ----- Ввод 'B'
22 mov ecx,B
23 mov edx,10
24 call sread
25
26 ; ----- Преобразование 'B' из символа в число
27 mov eax,B
28 call atoi ; Вызов подпрограммы перевода символа в число
29 mov [B],eax ; запись преобразованного числа в 'B'
30
31 ; ----- Записываем 'A' в переменную 'max'
32 mov ecx,[A] ; 'ecx = A'
33 mov [max],ecx ; 'max = A'
34
35 ; ----- Сравниваем 'A' и 'C' (как символы)
36 cmp ecx,[C] ; Сравниваем 'A' и 'C'
37 jg check_B ; если 'A>C', то переход на метку 'check_B',
38 mov ecx,[C] ; иначе 'ecx = C'
39 mov [max],ecx ; 'max = C'
40
41 ; ----- Преобразование 'max(A,C)' из символа в число
42 check_B:
43 mov eax,
44 call atoi ; Вызов подпрограммы перевода символа в число
45 mov [max],eax ; запись преобразованного числа в 'max'
abnikilaeva@fedora:~/work/arch-pc/lab07$

```

Рис. 2.11: Удаление операнда из программы

В новом файле листинга показывает ошибку, которая возникла при попытке трансляции файла. Никакие выходные файлы при этом помимо файла листинга не создаются. (рис. 2.12).

```

~/work/arch-pc/lab07/lab7-2.lst - Mousepad
x

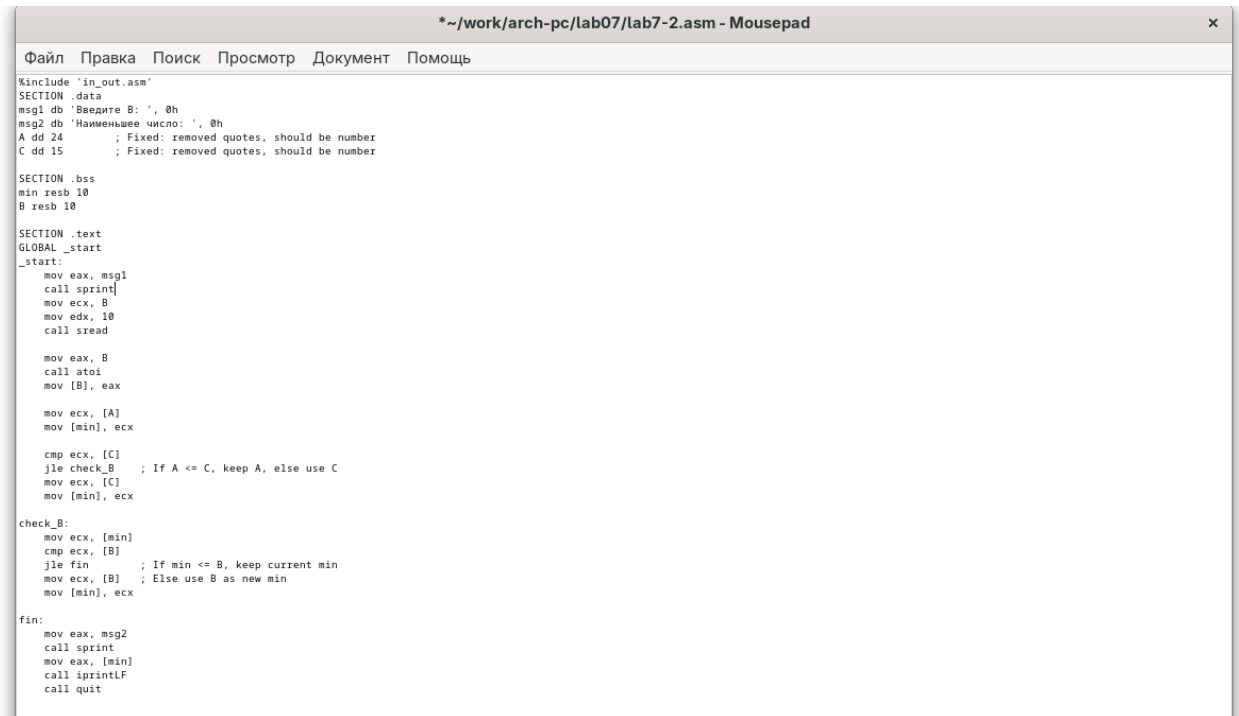
Файл  Правка  Поиск  Просмотр  Документ  Помощь
28 00000106 E891FFFFFF call atoi ; Вызов подпрограммы перевода символа в число
29 0000010B A3[0A000000] mov [B],eax ; запись преобразованного числа в 'B'
30
31 ; ----- Записываем 'A' в переменную 'max'
32 00000110 8B0D[35000000] mov ecx,[A] ; 'ecx = A'
33 00000116 8B0D[00000000] mov [max],ecx ; 'max = A'
34
35 ; ----- Сравниваем 'A' и 'C' (как символы)
36 0000011C 3B0D[39000000] cmp ecx,[C] ; Сравниваем 'A' и 'C'
37 00000122 7F0C jg check_B ; если 'A>C', то переход на метку 'check_B',
38 00000124 8B0D[39000000] mov ecx,[C] ; иначе 'ecx = C'
39 0000012A 8B0D[00000000] mov [max],ecx ; 'max = C'
40
41 ; ----- Преобразование 'max(A,C)' из символа в число
42 check_B:
43 mov eax,
44 ***** error: invalid combination of opcode and operands *****
45 00000130 E867FFFFFF call atoi ; Вызов подпрограммы перевода символа в число
46 00000135 A3[00000000] mov [max],eax ; запись преобразованного числа в 'max'
47
48 ; ----- Сравниваем 'max(A,C)' и 'B' (как числа)
49 0000013A 8B0D[00000000] mov ecx,[max]
50 00000140 3B0D[0A000000] cmp ecx,[B] ; Сравниваем 'max(A,C)' и 'B'
51 00000146 7F0C jg fin ; если 'max(A,C)>B', то переход на 'fin',
52 00000148 8B0D[0A000000] mov ecx,[B] ; иначе 'ecx = B'
53 0000014E 8B0D[00000000] mov [max],ecx
54 ; ----- Вывод результата
55 fin:
56 00000154 B8[13000000] mov eax,msg2
57 00000159 E801FFFFFF call sprint ; Вывод сообщения 'Наибольшее число: '
58 0000015E A1[00000000] mov eax,[max]
59 00000163 E81EFFFFFF call iprintf ; Вывод 'max(A,B,C)'
60 00000168 E86EFFFFFF call quit ; Выход

```

Рис. 2.12: Просмотр ошибки в файле листинга

2.3. Задания для самостоятельной работы

Делаю девятый вариант - из предыдущей лабораторной работы. Возвращаю операнд к функции в программе и изменяю ее так, чтобы она выводила переменную с наименьшим значением (рис. 2.13).



```
*~/work/arch-pc/lab07/lab7-2.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

%include 'in_out.asm'
SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'Наименьшее число: ', 0h
A dd 24      ; Fixed: removed quotes, should be number
C dd 15      ; Fixed: removed quotes, should be number

SECTION .bss
min resb 10
B resb 10

SECTION .text
GLOBAL _start
_start:
    mov eax, msg1
    call sprintf
    mov ecx, B
    mov edx, 10
    call read

    mov eax, B
    call atoi
    mov [B], eax

    mov ecx, [A]
    mov [min], ecx

    cmp ecx, [C]
    jle check_B ; If A <= C, keep A, else use C
    mov ecx, [C]
    mov [min], ecx

check_B:
    mov ecx, [min]
    cmp ecx, [B]
    jle fin ; If min <= B, keep current min
    mov ecx, [B] ; Else use B as new min
    mov [min], ecx

fin:
    mov eax, msg2
    call sprintf
    mov eax, [min]
    call fprintf
    call quit
```

Рис. 2.13: Первая программа самостоятельной работы

Код первой программы:

```
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'Наименьшее число: ', 0h
A dd '24'
C dd '15'

SECTION .bss
min resb 10
B resb 10

SECTION .text
GLOBAL _start
```

```

_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [min], ecx

cmp ecx, [C]
jle check_B
mov ecx, [C]

mov [min], ecx

check_B:
mov eax, [min]
cmp ecx, [B]
jle fin
mov ecx, [B]
mov [min], eax

fin:
mov eax, msg2
call sprint

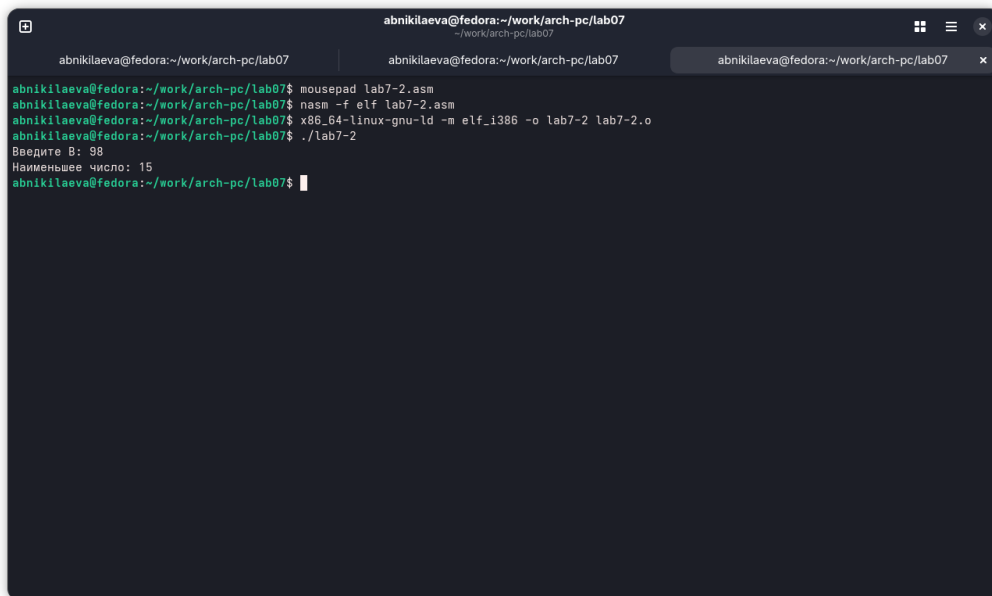
mov eax, [min]

call iprintLF

call quit

```

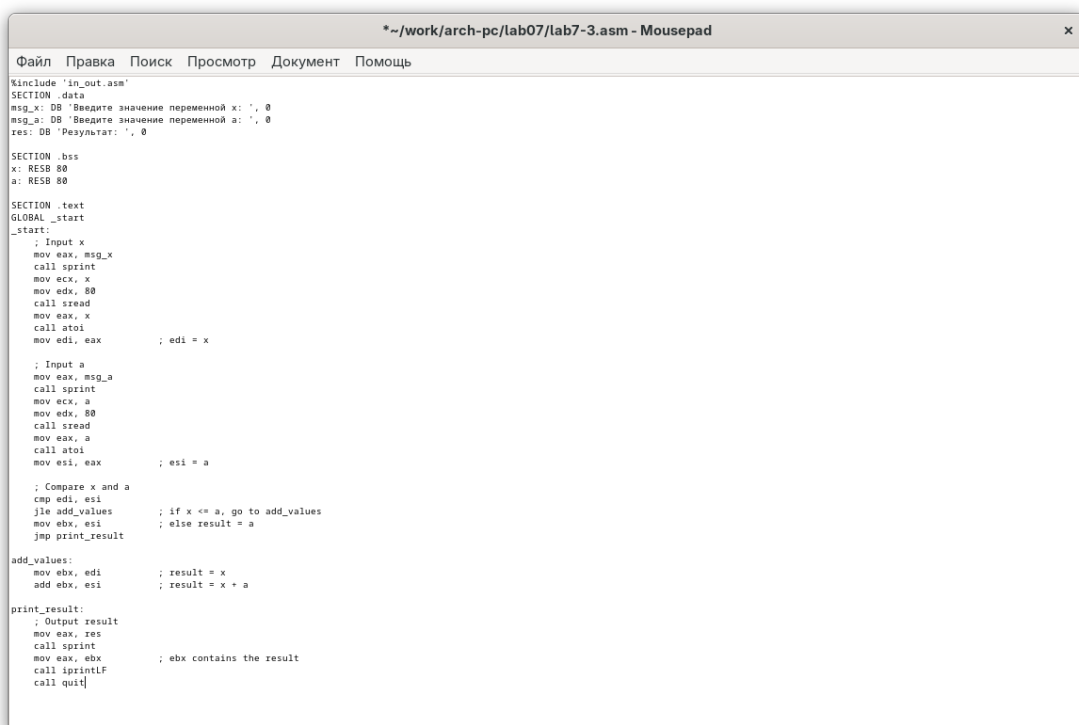
Проверяю корректность написания первой программы (рис. 2.14).



```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ mousepad lab7-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-2.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ x86_64-linux-gnu-ld -m elf_i386 -o lab7-2 lab7-2.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-2
Введите B: 98
Наименьшее число: 15
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Рис. 2.14: Проверка работы первой программы

Пишу программу, которая будет вычислять значение заданной функции согласно моему варианту для введенных с клавиатуры переменных а и х (рис. 2.15).



```
*~/work/arch-pc/lab07/lab7-3.asm - Mousepad
Файл  Правка  Поиск  Просмотр  Документ  Помощь

#include "in_out.asm"
SECTION .data
msg_x: DB "Введите значение переменной x: ", 0
msg_a: DB "Введите значение переменной a: ", 0
res: DB "Результат: ", 0

SECTION .bss
x: RESB 80
a: RESB 80

SECTION .text
GLOBAL _start
_start:
    ; Input x
    mov eax, msg_x
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    mov edi, eax        ; edi = x

    ; Input a
    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 80
    call sread
    mov eax, a
    call atoi
    mov esi, eax        ; esi = a

    ; Compare x and a
    cmp edi, esi
    jle add_values      ; if x <= a, go to add_values
    mov ebx, esi        ; else result = a
    jmp print_result

add_values:
    mov ebx, edi        ; result = x
    add ebx, esi        ; result = x + a

print_result:
    ; Output result
    mov eax, res
    call sprint
    mov eax, ebx
    call iprintf        ; ebx contains the result
    call quit
```

Рис. 2.15: Вторая программа самостоятельной работы

Код второй программы:

```
%include 'in_out.asm'

SECTION .data
msg_x: DB 'Введите значение переменной x: ', 0 msg_a: DB
'Введите значение переменной a: ', 0 res: DB
'Результат: ', 0

SECTION .bss
x: RESB 80

a: RESB 80

SECTION .text
GLOBAL _start
_start:
mov eax, msg_x
call sprint
mov ecx, x
mov edx, 80
call sread
mov eax, x
call atoi mov
edi, eax

mov eax, msg_a
call sprint mov
ecx, a

mov edx, 80
call sread
mov eax, a
call atoi mov
esi, eax

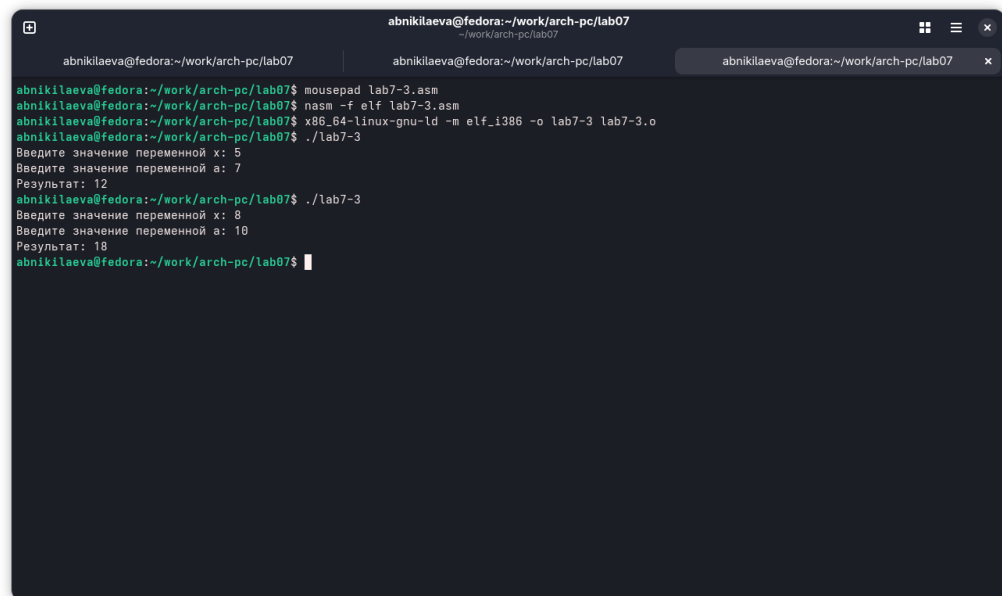
cmp edi, esi jle
add_values mov
eax, esi jmp
print_result

add_values:
mov eax, edi
add eax, esi
```

```
print_result:
mov edi, res
call sprint

mov eax, edi
call iprintLF
call quit
```

Транслирую и компоную файл, запускаю и проверяю работу программы для различных значений а и х (рис. 2.16).



```
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07
abnikilaeva@fedora:~/work/arch-pc/lab07$ mousepad lab7-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ nasm -f elf lab7-3.asm
abnikilaeva@fedora:~/work/arch-pc/lab07$ x86_64-linux-gnu-ld -m elf_i386 -o lab7-3 lab7-3.o
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-3
Введите значение переменной х: 5
Введите значение переменной а: 7
Результат: 12
abnikilaeva@fedora:~/work/arch-pc/lab07$ ./lab7-3
Введите значение переменной х: 8
Введите значение переменной а: 10
Результат: 18
abnikilaeva@fedora:~/work/arch-pc/lab07$
```

Рис. 2.16: Проверка работы второй программы

3.Выводы

При выполнении лабораторной работы я изучил команды условных и безусловных переходов, а также приобрел навыки написания программ с использованием переходов, познакомился с назначением и структурой файлов листинга.